

2026학년도 2학기 언어데이터과학

제4강 AWK 기초

박수민

서울대학교 인문대학 언어학과

2026년 9월 14일 월요일

지난 시간에 배운 것

- 1 파일을 열지 않고 살펴보기: `wc`, `head`, `tail`, `less`
- 2 명령 잇기와 결과 저장하기: `|`, `>`
- 3 `grep`과 정규표현식으로 찾고 세기: `sort`, `uniq -c`

오늘의 목표

- 1 텍스트를 레코드와 필드로 나누어 볼 수 있다.
- 2 FS와 RS를 바꾸어 내 자료에 맞는 단위를 지정할 수 있다.
- 3 AWK 스크립트를 파일로 작성하고 실행할 수 있다.
- 4 요리책 한 권을 레시피 단위로 쪼개고, 레시피 단위로 분석할 수 있다.

사전 준비

터미널 입력

```
cd /workspaces/LDS2026  
ls data/apicius
```

실행 결과

```
book.txt  recipes.txt  word-freq.txt
```

오늘은 이 세 파일 위에서 시작한다. 새로 내려받을 것은 없다.

3강을 마치지 못했다면

터미널 입력: 3강의 결과를 한꺼번에 다시 만들기

```
cd /workspaces/LDS2026
mkdir -p data/apicius
cd data/apicius
wget https://www.gutenberg.org/cache/epub/29728/pg29728.txt -O book.txt
head -n 13700 book.txt | tail -n +2938 > recipes.txt
grep -oE "[A-Z][A-Z][A-Z]+" recipes.txt | sort | uniq -c | sort -nr >
↪ word-freq.txt
cd /workspaces/LDS2026
```

여섯 줄로 복구된다는 것

3강에서 며칠 걸릴 일을 명령으로 남겨 두었기 때문이다. 데이터가 아니라 **데이터를 만든 명령**을 남기면 언제든지 같은 자리로 돌아올 수 있다.

1 행에서 레코드로 3강이 남긴 문제 AWK

2 AWK 기본 문법

3 구분자 바꾸기

4 레시피 단위로 분석하기

5 레시피 카드 만들기

6 마무리

대답하지 못한 질문

3강에서 알아낸 것

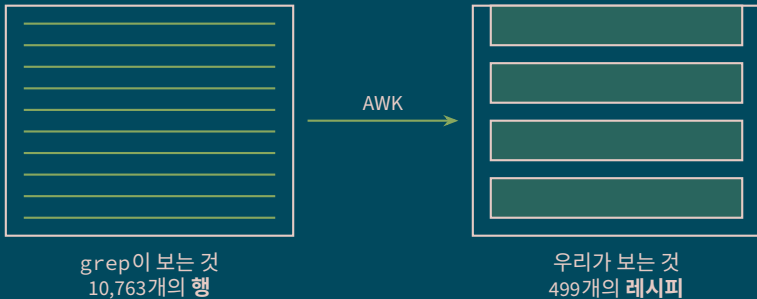
PEPPER는 467번, HONEY는 226번 나온다. PEPPER와 HONEY가 같은 **행**에 있는 곳은 75군데다.

그런데 정작 궁금한 것

- 후추를 쓰는 **요리**는 몇 가지인가?
- 후추와 꿀을 **함께** 쓰는 요리는 얼마나 되는가?
- 이 책에 실린 요리는 모두 몇 개인가?

한 레시피는 여러 행에 걸쳐 있다. 행을 세는 것으로는 요리를 셀 수 없다.

같은 파일, 두 가지 눈



오늘 하는 일

파일은 그대로 두고, 읽는 단위를 바꾼다.

AWK

AWK

1977년 벨 연구소에서 만들어진, 텍스트를 줄 단위로 훑으며 처리하는 언어.

이름

만든 세 사람의 성을 딴 것이다.

Alfred Aho 뒤에 The AWK Programming Language 공저

Peter Weinberger 통계 처리 기능 설계

Brian Kernighan C 언어 교과서의 그 커니한

50년 가까이 살아남은 이유

어느 리눅스에도나 설치되어 있고, 짧은 한 줄로 표 형식 텍스트를 다룰 수 있다.

grep, sed, awk

도구	하는 일	이 수업에서
grep	조건에 맞는 행을 고른다	3강: 용례 찾기
sed	문자열을 바꾼다	(다루지 않는다)
awk	행을 나누어 계산하고 만든다	4강: 구조 부여하기

결정적인 차이

grep에게 행은 **쪼갤 수 없는 덩어리**다. AWK에게 행은 **여러 칸으로 나뉜 자료**이고, 그 칸을 꺼내 세고 계산할 수 있다.

그리고 하나 더

AWK는 “무엇을 한 덩어리로 볼 것인가”를 우리가 정할 수 있다.

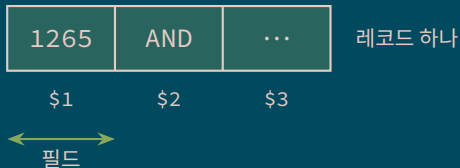
레코드와 필드

두 개의 단위

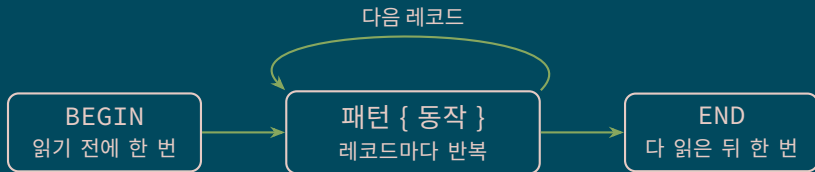
레코드 한 번에 처리하는 한 덩어리
(기본값: 한 행)

필드 레코드를 다시 나눈 칸
(기본값: 빈칸으로 구분)

AWK는 파일을 **레코드로 잘라** 하나씩
읽고, 그 레코드를 다시 **필드로**
나눈다.



AWK가 파일을 읽는 순서



세 자리의 쓰임

BEGIN 구분자 설정, 머리글 출력, 변수 초기화

패턴 { 동작 } 조건에 맞는 레코드에 대해 무엇을 할지

END 합계·개수처럼 **다 읽어야 알 수 있는 것** 출력

1 행에서 레코드로

2 **AWK 기본 문법**

패턴과 동작
필드 꺼내기
고르고 세기

3 구분자 바꾸기

4 레시피 단위로 분석하기

5 레시피 카드 만들기

6 마무리

명령의 구조

awk	'\$1 >= 100	{ print \$2 }'	word-freq.txt
	↑	↑	↑
	패턴	동작	인자
	어느 레코드에	무엇을 할지	어느 파일에서

규칙

- AWK 코드 전체를 **작은따옴표**로 감싼다. (셀이 \$를 가로채지 않도록)
- 동작은 **중괄호** 안에 적는다.
- 읽는 법: “패턴에 맞는 레코드마다 동작을 하라”

패턴과 동작은 둘 다 생략할 수 있다

적는 방식	뜻	예
패턴 {동작}	맞는 레코드에만 동작	<code>\$1 >= 100 { print \$2 }</code>
{동작}	모든 레코드에 동작	<code>{ print \$2 }</code>
패턴	맞는 레코드를 그대로 출력	<code>\$1 >= 100</code>
BEGIN {동작}	시작할 때 한 번	<code>BEGIN { FS = "," }</code>
END {동작}	끝날 때 한 번	<code>END { print NR }</code>

기억할 것

`grep "WINE" recipes.txt`와 `awk '/WINE/' recipes.txt`는 거의 같은 일을 한다. AWK는 여기에서 **한 걸음 더** 나갈 수 있을 뿐이다.

연습할 자료: 3강에서 만든 빈도표

터미널 입력

```
head -n 3 data/apicius/word-freq.txt
```

실행 결과

```
1265 AND  
1009 THE  
726 WITH
```

이 파일의 구조

한 행이 한 레코드이고, 빈칸으로 나뉜 두 개의 필드가 있다. \$1은 빈도, \$2는 단어다.

필드 하나만 꺼내기

터미널 입력

```
awk '{ print $2 }' data/apicius/word-freq.txt | head -n 5
```

실행 결과

```
AND  
THE  
WITH  
PEPPER  
BROTH
```

패턴을 적지 않았으므로 모든 레코드에 대해 실행된다. AWK의 출력도 파이프로 넘길 수 있다.

\$0과 \$NF

표기	가리키는 것
\$0	레코드 전체 (자른 적이 없는 원래 모습)
\$1, \$2, ...	첫 번째, 두 번째 필드
NF	이 레코드의 필드 개수 (number of fields)
\$NF	마지막 필드
NR	지금까지 읽은 레코드 번호 (number of records)

\$가 붙고 안 붙고

NF는 개수 2이고, \$NF는 \$2, 즉 그 자리의 **값**이다. NR과 \$NR을 헷갈리는 것이 가장 흔한 실수다.

심표를 빠뜨리면

터미널 입력

```
awk 'NR <= 3 { print $2, $1 }' data/apicius/word-freq.txt  
awk 'NR <= 3 { print $2 $1 }' data/apicius/word-freq.txt
```

```
AND 1265  
THE 1009  
WITH 726
```

심표: 사이에 빈칸을 넣어 출력

NR <= 3이 패턴, { print ...}이 동작이다. 세 줄만 보고 싶을 때 쓴다.

```
AND1265  
THE1009  
WITH726
```

심표 없음: 그냥 이어 붙임

순위를 붙여 보기

터미널 입력

```
awk 'NR <= 5 { print NR, $2, $1 }' data/apicius/word-freq.txt
```

실행 결과

```
1 AND 1265  
2 THE 1009  
3 WITH 726  
4 PEPPER 467  
5 BROTH 400
```

파일은 이미 빈도순으로 정렬되어 있으므로 NR이 곧 순위가 된다. 3강의 sort가 여기에서 값을 한다.

조건으로 고르기

터미널 입력

```
awk '$1 >= 100 { n++ } END { print n }' data/apicius/word-freq.txt
```

실행 결과

32

읽는 법

\$1 >= 100 첫 필드를 수로 보고 비교한다

n++ n을 1 늘린다. 선언할 필요도, 0으로 시작할 필요도 없다

END { ... } 다 읽은 뒤에 한 번 출력한다

100번 이상 나온 단어는 2,434개 중 32개뿐이다.

정규표현식으로 고르기

터미널 입력

```
awk '$2 ~ /^PEPPER/ { print }' data/apicius/word-freq.txt
```

실행 결과

```
467 PEPPER  
1 PEPPERS  
1 PEPPERCORNS
```

읽는 법

~는 “왼쪽이 오른쪽 패턴에 맞는가”이다. (맞지 않는 것을 고르려면 !~) print는 인자 없이 쓰면 \$0, 즉 레코드 전체를 출력한다.

모두 더하기

터미널 입력

```
awk '{ total += $1 } END { print NR, total }' data/apicius/word-freq.txt
```

실행 결과

```
2434 21328
```

두 숫자의 이름

2434 서로 다른 단어의 수

21328 그 단어들이 나타난 횟수의 합

하나는 “몇 종류인가”이고 하나는 “몇 번인가”이다. 이 구별에는 이름이 있다(타입과 토큰). 8강에서 본격적으로 다룬다.

확인 질문

여기까지 정리

- 1 `awk '{ print $1 }'`와 `awk '{ print $0 }'`의 차이는?
- 2 `awk 'NR == 10'`은 무엇을 출력하는가?
- 3 `awk 'END { print NR }'`와 `wc -l`은 각각 무엇을 세는가?
- 4 `$1 >= 100`에서 `$1`이 "1265"라는 글자가 아니라 수 1265로 다루어지는 이유는?

1 행에서 레코드로

2 AWK 기본 문법

3 **구분자 바꾸기**
필드 구분자 FS
레코드 구분자 RS

4 레시피 단위로 분석하기

5 레시피 카드 만들기

6 마무리

쉼표로 나뉜 자료 만들기

터미널 입력

```
echo 'PEPPER,후추,467' > data/apicius/ingredients.csv  
echo 'BROTH,육수,400' >> data/apicius/ingredients.csv  
echo 'WINE,포도주,370' >> data/apicius/ingredients.csv  
echo 'HONEY,꿀,226' >> data/apicius/ingredients.csv  
cat data/apicius/ingredients.csv
```

실행 결과

```
PEPPER,후추,467  
BROTH,육수,400  
WINE,포도주,370  
HONEY,꿀,226
```

>는 새로 쓰고 >>는 이어 쓴다. 3장에서 본 그 기호다.

기본 구분자로는 나뉘지 않는다

터미널 입력

```
awk '{ print $2 }' data/apicius/ingredients.csv  
awk '{ print NF }' data/apicius/ingredients.csv | head -n 1
```

실행 결과

(빈 줄 네 개)
1

왜 그런가

기본 필드 구분자는 **빈칸**이다. 이 파일에는 빈칸이 없으므로 한 행 전체가 \$1 하나이고, \$2는 비어 있다.

FS: 필드 구분자

터미널 입력

```
awk -F',' '{ print $2 }' data/apicius/ingredients.csv  
awk -F',' '$3 >= 300 { print $2 }' data/apicius/ingredients.csv
```

후추
육수
포도주
꿀

첫 번째 명령: 모든 행

후추
육수
포도주

두 번째 명령: 300 이상만

-F', ' 는 “필드를 쉼표로 나누라”는 뜻이다. 이제 \$3으로 조건을 걸 수 있다.

OFS: 출력 필드 구분자

터미널 입력

```
awk 'BEGIN { FS = ","; OFS = " - " } { print $2, $1 }'  
↪ data/apicius/ingredients.csv
```

실행 결과

```
후추 - PEPPER  
육수 - BROTH  
포도주 - WINE  
꿀 - HONEY
```

읽는 법

BEGIN에서 설정하면 -F 옵션과 같은 일을 한다. 여러 개는 ;로 잇는다. OFS는 print의 **심표 자리**에 끼워 넣을 문자열이다.

네 개의 구분자

변수	무엇을 나누는가	기본값	방향
FS	필드	빈칸	입력
RS	레코드	줄바꿈	입력
OFS	필드	빈칸 한 칸	출력
ORS	레코드	줄바꿈	출력

O는 output

FS/RS는 “어떻게 읽을까”, OFS/ORS는 “어떻게 써낼까”이다.

오늘의 열쇠

RS를 바꾸면 “한 레코드”의 뜻 자체가 달라진다. 여기에서 AWK가 grep과 갈라진다.

요리책의 구조

한 레시피의 생김새

- 1 제목 ([번호]로 시작, 두 줄)
- 2 본문
- 3 각주 (없을 수도, 여러 개일 수도)

빈 줄의 규칙

- 덩어리 사이: 빈 줄 **1개**
- 레시피 사이: 빈 줄 **2개 이상**

[1] FINE SPICED WINE

빈 줄 1개

본문

빈 줄 1개

각주 [1] [2] [3]

빈 줄 2개

[2] HONEY REFRESHER

그 전에: 줄바꿈에도 방언이 있다

터미널 입력: `cat -A`는 보이지 않는 문자까지 보여준다

```
head -n 2 data/apicius/recipes.txt | cat -A
```

실행 결과

```
[1] FINE SPICED WINE^M$  
_CONDITUM PARADOXUM_^M$
```

읽는 법

\$ 행의 끝 (줄바꿈 LF, `\n`)

^M 그 앞에 하나 더 있는 문자 (CR, `\r`)

Windows에서 만든 텍스트는 행 끝에 CR LF 두 글자를 쓴다. 이 파일이 그렇다.

CR 걷어내기

터미널 입력

```
tr -d '\r' < data/apicius/recipes.txt > data/apicius/recipes-lf.txt  
wc data/apicius/recipes.txt data/apicius/recipes-lf.txt
```

실행 결과

```
10763    57048   367682 data/apicius/recipes.txt  
10763    57048   356919 data/apicius/recipes-lf.txt
```

읽는 법

tr -d '\r'는 CR를 지운다. <는 파일을 명령의 입력으로 넣는 기호다. 행 수와 단어 수는 그대로, 바이트만 10,763만큼 줄었다.

원본은 고치지 않고 새 파일로 만들었다. 3강의 원칙 그대로다.

RS 바꾸기

터미널 입력

```
awk 'BEGIN { RS = "\n\n\n" } END { print NR }' \
data/apicius/recipes-lf.txt
```

실행 결과

```
731
```

읽는 법

\n\n\n+는 줄바꿈 세 개 이상, 즉 **빈 줄 두 개 이상**이다. 3강에서 배운 정규표현식을 RS 에도 쓸 수 있다. 10,763행이던 파일이 731개의 레코드가 되었다. 파일이 아니라 **보는 눈**이 바뀐 것이다.

FS도 함께 바꾸기

터미널 입력

```
awk 'BEGIN { RS = "\n\n\n"; FS = "\n\n" } NR <= 3 { print NR, NF }' \
data/apicius/recipes-lf.txt
```

실행 결과

```
1 7
2 4
3 1
```

읽는 법

레코드 안에서는 빈 줄 하나(`\n\n`)가 필드를 나눈다. \$1은 제목, \$2는 본문, \$3부터는 각주다. 첫 레시피는 필드가 7개, 즉 각주가 5개다.

1 행에서 레코드로

2 AWK 기본 문법

3 구분자 바꾸기

4 **레시피 단위로 분석하기**

레시피만 고르기

스크립트로 옮기기

표로 만들기

5 레시피 카드 만들기

6 마무리

731개는 레시피가 아니다

터미널 입력

```
awk 'BEGIN { RS = "\n\n\n+"; FS = "\n\n" } { print NR": "$1 }' \
data/apicius/recipes-lf.txt | head -n 9
```

실행 결과

```
1: [1] FINE SPICED WINE
   _CONDITUM PARADOXUM_
2: [2] HONEY REFRESHER FOR TRAVELERS
   _CONDITUM MELIZOMUM _[1]_ VIATORIUM_
3: II
4: [3] ROMAN VERMOUTH
   _ABSINTHIUM ROMANUM_ [1]
5: III
```

권 번호(II, III)가 섞여 있다. 레시피의 제목은 언제나 [숫자]로 시작한다.

패턴으로 레시피만 세기

터미널 입력

```
awk 'BEGIN { RS = "\n\n\n+"; FS = "\n\n" } $1 ~ /^\[ [0-9]+\]/ { n++ }  
END { print n }' data/apicius/recipes-lf.txt
```

실행 결과

499

패턴 읽기: `^\[ [0-9]+\]`

<code>^</code> 시작이	<code>[0-9]+</code> 숫자 한 자리 이상
<code>\[</code> 대괄호 <code>[</code> (특수문자를 막는 <code>\</code>)	<code>\]</code> 닫는 대괄호

이 책에 실린 요리는 **499개**다. 3강에서 못 센 것을 이제 썼다.

3강의 질문에 답하기

터미널 입력

```
awk 'BEGIN { RS = "\n\n\n+"; FS = "\n\n" }  
$1 ~ /^\[0-9]+\[/ && $2 ~ /PEPPER/ && $2 ~ /HONEY/ { n++ }  
END { print n }' data/apicius/recipes-lf.txt
```

실행 결과

162

방법	세는 단위	값
3강 grep "PEPPER" grep "HONEY"	행	75
4강 AWK	레시피	162 / 499

후추와 꿀을 함께 쓰는 요리가 세 개 중 하나꼴이라는 것은 행을 세어서는 알 수 없었다.

코드가 길어지면 파일로

터미널 입력

```
mkdir -p awk  
code awk/count.awk      # VS Code에서 새 파일 열기
```

실행 방법

```
awk -f awk/count.awk data/apicius/recipes-lf.txt
```

-f는 “AWK 코드를 이 파일에서 읽으라”는 뜻이다.

왜 파일로 남기는가

- 터미널에 친 명령은 사라지지만, 파일은 저장소에 커밋된다.
- data/는 .gitignore에 있고 awk/는 아니다. **데이터가 아니라 방법을 남긴다.**

awk/count.awk: 재료별 레시피 수

```
1 BEGIN {
2     RS = "\n\n\n"
3     FS = "\n\n"
4     n = split("PEPPER OIL BROTH WINE HONEY VINEGAR CUMIN", spice, " ")
5 }
6 $1 ~ /^\[([0-9]+)\]/ {
7     total++
8     for (i = 1; i <= n; i++)
9         if ($2 ~ spice[i]) count[spice[i]]++
10 }
11 END {
12     for (i = 1; i <= n; i++) {
13         percent = 100 * count[spice[i]] / total
14         printf "%-9s %3d (%4.1f%%)\n", spice[i], count[spice[i]], percent
15     }
16     print "전체 레시피", total
17 }
```


코드 읽기

새로 나온 것

split(문자열, 배열, 구분자) 문자열을 잘라 배열에 담고 개수를 돌려준다

배열 `spice[1], spice[2], ...` 처럼 번호나 이름으로 값을 담는 상자

count[**spice**[**i**]] 이름표가 "PEPPER"인 칸. 없던 칸은 자동으로 생긴다

for 시작값, 계속할 조건, 한 번 돌 때마다 할 일을 차례로 적는다

printf 자리를 맞추어 출력한다. `%-9s` 왼쪽 정렬 9칸 문자열, `%3d` 3칸 정수, `%4.1f` 소수점 한 자리

눈여겨볼 곳

세는 일은 레코드마다 하고, 나누는 일(/ `total`)은 END에서 한다. 전체 개수는 끝까지 읽어야 알 수 있기 때문이다.

실행 결과

터미널 입력

```
awk -f awk/count.awk data/apicius/recipes-lf.txt
```

실행 결과

PEPPER	364	(72.9%)
OIL	350	(70.1%)
BROTH	317	(63.5%)
WINE	241	(48.3%)
HONEY	193	(38.7%)
VINEGAR	172	(34.5%)
CUMIN	116	(23.2%)
전체 레시피	499	

“467번 나온다”가 아니라 “**499개 요리 중 364개에 들어간다**”고 말할 수 있게 되었다.

로마 요리의 재료 (레시피 기준)



3강의 그림과 비교하면

3강에서 BROTH(400회)가 OIL(338회)보다 앞섰지만, **요리 수**로 세면 기름이 두 번째다. 육수는 한 레시피에서 여러 번 언급될 뿐이다.

awk/index.awk: 레시피 목록 만들기

```
1 BEGIN {
2     RS = "\n\n\n"
3     FS = "\n\n"
4     OFS = "\t" # 출력은 탭으로 구분
5     print "number", "title", "words" # 머리글
6 }
7 $1 ~ /^\[0-9\]+\[/ {
8     match($1, /[0-9]+/) # 제목에서 번호 찾기
9     number = substr($1, RSTART, RLENGTH) # 찾은 자리를 잘라내기
10    split($1, line, "\n") # 제목을 줄 단위로 나누기
11    title = line[1]
12    sub(/^\[0-9\]+\[/, "", title) # 앞의 [번호] 떼어내기
13    body = $2
14    gsub(/\n/, " ", body) # 본문을 한 줄로 잇기
15    print number, title, split(body, w, " ") # 번호, 제목, 단어 수
16 }
```

문자열을 다루는 함수

함수	하는 일
<code>match(문자열, /패턴/)</code>	패턴이 맞는 자리를 찾아 RSTART, RLENGTH에 넣는다
<code>substr(문자열, 시작, 길이)</code>	그 자리를 잘라낸다
<code>split(문자열, 배열, 구분자)</code>	잘라서 배열에 담고 개수 를 돌려준다
<code>sub(/패턴/, "바꿀 것", 대상)</code>	처음 맞는 곳 하나만 바꾼다
<code>gsub(/패턴/, "바꿀 것", 대상)</code>	맞는 곳을 모두 바꾼다 (g: global)
<code>length(문자열)</code>	글자 수를 센다

빈 문자열로 바꾸면 지우는 것이다

`sub(/^[0-9]+\ */, "", title)`은 "[97] BEETS"를 "BEETS"로 만든다.

표 만들고 다시 읽기

터미널 입력

```
awk -f awk/index.awk data/apicius/recipes-lf.txt > data/apicius/index.tsv
head -n 4 data/apicius/index.tsv
awk -F'\t' 'NR > 1 { n++; sum += $3 } END { print n, sum / n }'
↵ data/apicius/index.tsv
```

실행 결과

number	title	words
1	FINE SPICED WINE	144
2	HONEY REFRESHER FOR TRAVELERS	82
3	ROMAN VERMOUTH	73
499	45.1242	

빈칸처럼 보이는 자리는 모두 탭이다. 줄글이 표가 되었고, 레시피 한 편은 평균 45단어다.

가장 긴 레시피는?

터미널 입력

```
awk -F'\t' 'NR > 1 && $3 > max { max = $3; name = $2 }  
END { print max, name }' data/apicius/index.tsv
```

실행 결과

```
273 PEAS [supreme style]
```

읽는 법

max는 처음에 비어 있고, 수와 비교하면 0으로 취급된다. 지금까지의 최댓값보다 크면 갈아 끼운다. 정렬하지 않고도 최댓값을 찾는 방법이다.

TSV(탭으로 나뉜 표)는 7강에서 pandas로 그대로 읽어 들일 수 있는 형식이다.

1 행에서 레코드로

2 AWK 기본 문법

3 구분자 바꾸기

4 레시피 단위로 분석하기

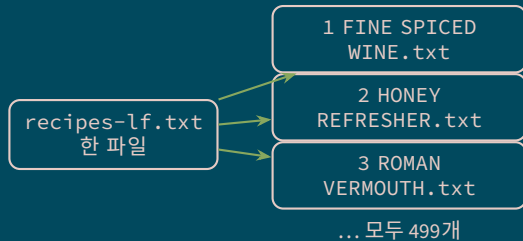
5 **레시피 카드 만들기**
설계
실행과 확인

6 마무리

마지막 목표

할 일

- 1 레시피 한 편을 파일 하나로 저장한다.
- 2 파일 이름은 **번호와 제목**으로 짓는다.
- 3 본문은 한 줄로 잇는다.



그러면 무엇이 좋은가

파일 이름이 곧 제목이므로, 3강에서 배운 `grep`만으로도 “어느 요리인지”를 알 수 있다.

awk/split-recipes.awk

```
1 BEGIN {
2     RS = "\n\n\n"
3     FS = "\n\n"
4 }
5 $1 ~ /^\[0-9]+\[/ {
6     match($1, /[0-9]+/)
7     number = substr($1, RSTART, RLENGTH)
8     split($1, line, "\n")
9     title = line[1]
10    sub(/^\[0-9]+\[/ *, "", title)
11    gsub(/^[A-Z ]/, "", title)
12    sub(/ +$/, "", title)
13    if (title == "") title = "UNNAMED"
14
15    body = $2
16    gsub(/\n/, " ", body)
17    file = "data/apicius/recipes/" number " " title ".txt"
18    print body > file
19    close(file)
20 }
```

레시피 번호

[번호] 떼어내기

파일 이름에 쓰기 어려운 글자 지우기

끝에 남은 빈칸 지우기

제목이 사라졌으면

본문을 한 줄로

화면이 아니라 파일로

다 쓴 파일은 닫는다

두 곳만 더 읽어 두기

```
print body > file
```

3장의 리다이렉션과 같은 기호지만, 여기에서는 **AWK 안에서** 파일을 정한다. `file`의 값이 레코드마다 달라지므로 파일이 499개 만들어진다.

```
gsub(/^[^A-Z ]/, "", title)
```

`[^...]`는 “괄호 안의 것이 **아닌** 문자”라는 뜻이다. 대문자와 빈칸만 남기고 `[1]`, 따옴표, / 같은 글자를 지운다.

- /가 이름에 들어가면 디렉터리로 해석되어 오류가 난다.

```
close(file)
```

열어 둔 파일이 수백 개가 되면 오류가 날 수 있다. 다 쓴 파일은 닫아 준다.

실행하기

터미널 입력

```
mkdir -p data/apicius/recipes
awk -f awk/split-recipes.awk data/apicius/recipes-lf.txt
ls data/apicius/recipes | wc -l
ls data/apicius/recipes | head -n 5
```

실행 결과

```
499
1 FINE SPICED WINE.txt
10 TO KEEP MEATS FRESH WITHOUT SALT FOR ANY LENGTH OF TIME.txt
100 TURNIPS OR NAVIEWS.txt
101 ANOTHER WAY.txt
102 RADISHES.txt
```

앞서 AWK가 센 499와 파일 개수가 같다. 두 방법이 같은 값을 낸다는 것은 좋은 신호다.

만들어진 카드 열어 보기

터미널 입력

```
cat "data/apicius/recipes/97 BEETS.txt"
```

실행 결과

```
TO MAKE A DISH OF BEETS THAT WILL APPEAL TO YOUR TASTE [1] SLICE [the beets, [2]  
with] LEEKS AND CRUSH CORIANDER AND CUMIN; ADD RAISIN WINE [3], BOIL ALL DOWN TO  
PERFECTION: BIND IT, SERVE [the beets] SEPARATE FROM THE BROTH, WITH OIL AND  
VINEGAR.
```

파일 이름에 빈칸이 있으므로 **따옴표**로 감싸야 한다. (97 뒤에서 **Tab**을 누르면 나머지를 채워 준다.)

카드로 다시 검색하기

터미널 입력

```
grep -l "PEPPER" data/apicius/recipes/*.txt | wc -l  
grep -l "CABBAGE" data/apicius/recipes/*.txt | head -n 3
```

실행 결과

```
364  
data/apicius/recipes/103 SOFT CABBAGE.txt  
data/apicius/recipes/140 A RICH ENTRE OF FISH POULTRY AND SAUSAGE IN CREAM.txt  
data/apicius/recipes/173 ANOTHER TISANA.txt
```

읽는 법

-l은 맞는 행이 아니라 **파일 이름**을 출력한다. 364는 count.awk가 센 값과 같다.

무엇이 달라졌는가

3강까지

- 파일 = 행의 나열
- 셀 수 있는 것: 단어와 행
- “어느 요리인지”는 사람이 눈으로 찾아야 했다

지금

- 파일 = 레시피 499개
- 셀 수 있는 것: 요리, 재료, 길이
- 이름과 번호가 붙은 **데이터**가 되었다

남은 흠

ENTRÉE가 ENTRE가 되었다. [^A-Z]가 é를 지웠기 때문이다.

- 영어 대문자만 “글자”로 본 셈이다. 한국어 자료였다면 전부 지워졌을 것이다.
- 왜 그런지, 어떻게 고치는지는 9강(문자 인코딩과 유니코드)에서 다룬다.

1 행에서 레코드로

2 AWK 기본 문법

3 구분자 바꾸기

4 레시피 단위로 분석하기

5 레시피 카드 만들기

6 **마무리**

요약

다음 시간 예고

과제

오늘 쓴 AWK

내장 변수

\$0 레코드 전체
\$1, \$NF 첫 필드, 마지막 필드
NR 레코드 번호
NF 필드 개수
FS, RS 입력 구분자
OFS, ORS 출력 구분자

구문과 함수

BEGIN 시작할 때 한 번
END 끝날 때 한 번
~ 정규표현식에 맞는가
if, for 조건과 반복
split 잘라서 배열에 담기
sub, gsub 바꾸기·지우기
match, substr 찾아서 잘라내기
printf 자리 맞추어 출력

자주 만나는 오류

증상	원인과 대처
아무것도 출력되지 않는다 빈 줄만 나온다 awk: syntax error 셀이 이상하게 해석한다 \$가 사라진다 레코드가 나뉘지 않는다	패턴에 맞는 레코드가 없다. 패턴을 빼고 확인한다 FS가 맞지 않아 \$2가 비어 있다 중괄호·따옴표의 짝을 확인한다 큰따옴표 대신 작은따옴표 로 감싼다 큰따옴표 안의 \$1은 셀이 먼저 가로챈다 CR가 남아 있다. <code>tr -d '\r'</code> 를 먼저 한다

습관

긴 스크립트는 한 번에 쓰지 않는다. `print $1`로 먼저 확인하고, 조건을 하나씩 붙인다.

확인 질문

정리

- 1 RS를 바꾸지 않으면 한 레코드는 무엇인가?
- 2 awk '{ print NF }'가 모두 1을 출력한다면 무엇을 의심해야 하는가?
- 3 같은 파일에서 grep -c는 468, AWK는 364를 세었다. 무엇이 다른가?
- 4 레시피가 아닌 레코드(II, III)를 걸러낸 패턴은 무엇이었는가?
- 5 sub와 gsub는 어떻게 다른가?

오늘 배운 것

- 1 AWK의 세계관: 레코드와 필드, BEGIN — 패턴 {동작} — END
- 2 구분자 바꾸기: FS, RS, OFS
- 3 스크립트 파일과 `awk -f`, 배열과 반복문으로 집계하기
- 4 요리책 한 권을 레시피 499개로 쪼개고 표로 만들기

다음 시간에 배울 것 (9월 16일 수요일)

- 1 데이터의 정의와 범주, 언어데이터의 범위
- 2 기술통계량 — 오늘 만든 “평균 45단어”가 무엇을 말해 주고 무엇을 감추는가

[숙제02] AWK 연습

할 일

awk/split-recipes.awk를 고쳐서 레시피 카드를 아래 형식으로 만든다.

- 1 파일 이름은 번호 - 영어 제목 - 라틴어 제목.txt 형식으로 한다.
- 2 1행에는 레시피 본문을 한 줄로 넣는다.
- 3 2행에는 -----를 넣는다.
- 4 3행부터 각주를 하나씩 한 행에 넣는다. (없으면 3행도 없다)
- 5 불필요한 줄바꿈과 들여쓰기가 남아 있으면 안 된다.

힌트

각주는 \$3부터 \$NF까지다. `for (i = 3; i <= NF; i++) { ... }`

라틴어 제목은 제목의 둘째 줄, 즉 `split($1, line, "\n")`의 `line[2]`에 있다.

[숙제02] 예상 결과

터미널 입력

```
cat "data/apicius/recipes/97 - BEETS - BETAS.txt"
```

```
TO MAKE A DISH OF BEETS THAT WILL APPEAL TO YOUR TASTE [1] SLICE [the beets, [2] with] LEEKS AND CRUSH  
↪ CORIANDER AND CUMIN; ADD RAISIN WINE [3], BOIL ALL DOWN TO PERFECTION: BIND IT, SERVE [the beets] SEPARATE  
↪ FROM THE BROTH, WITH OIL AND VINEGAR.
```

```
-----  
[1] Sentence in Tor.; wanting in List. _et al._
```

```
[2] List. No mention of beets is made in this formula; therefore, it may belong to the foregoing leek recipes.  
↪ V. This is not so. Here the noun is made subject to the first verb, as is practiced frequently. Moreover,  
↪ the mode of preparation fits beets nicely, except for the flour to which we object in note 3, below. To  
↪ cook beets with leeks, spices and wine and serve them (cold) with oil and vinegar is indeed a method that  
↪ cannot be improved upon.
```

```
[3] Tac., Tor., List., G.-V. _uvam passam_, _Farinam_--raisins and flour--for which there is no reason. Sch.  
↪ _varianam_--raisin wine of the Varianian variety; Bas. _Phariam_. V. inclined to agree with Sch. and Bas.
```

제출 방법

터미널 입력

```
cd /workspaces/LDS2026
git add awk/
git commit -m "4강 AWK 스크립트 추가"
git push
```

제출

- 제출 기한: 2026-09-18 12:00 (금요일 정오)
- 제출 방법: 저장소에 push하면 된다. eTL에 파일을 따로 낼 필요는 없다.
- 데이터 파일(data/)은 커밋하지 않는다. 스크립트만 남긴다.

막히면 eTL Q&A 게시판에 질문한다. (답변 마감: 목요일 12:00)